



## FORZA 4

Progetto Accademico Laboratorio di Sistemi Operativi

CERVERA FRANCESCO

CORRERA SALVATORE

## **1. Introduzione**

Il sistema implementa il gioco Forza 4 multiplayer via rete: più client si connettono a un server centrale, creando o unendosi a partite e giocando a turni con una rigorosa sincronizzazione dello stato. I principali vincoli progettuali affrontati sono stati la gestione sicura della concorrenza (con più stanze attive in parallelo), la consistenza di rete per garantire mosse valide e rigorosamente ordinate, e la tolleranza alle disconnessioni impreviste.

## **2. Architettura: Server Autoritativo e Client Leggero**

Abbiamo scelto un'architettura server autoritativa con client leggero. Tutta la logica matematica, la simulazione della gravità dei gettoni e la determinazione delle vittorie sono centralizzate nel server. Il client Java ha il solo ruolo di interfaccia grafica (GUI) asincrona per l'interazione visiva con l'utente.

Questa rigida separazione è stata adottata per mantenere uno stato unico e coerente del gioco: il server rappresenta l'unica fonte di verità, evitando alla radice problemi di desincronizzazione tra client. La scelta dei linguaggi segue la stessa logica: il server in C gestisce socket, concorrenza e risorse POSIX a basso livello, mentre il client in Java astrae la rete delegando la renderizzazione grafica alle librerie Swing.

### 3. Modello di Concorrenza

#### 3.1 Thread per client (Modello Detached)

Il server utilizza un thread POSIX dedicato per ogni connessione TCP, creato tramite la primitiva `pthread_create` e immediatamente svincolato dal thread principale tramite `pthread_detach`, evitando così la necessità di una `pthread_join`. Questo modello semplifica la gestione del flusso del client, permettendo a ogni thread di gestire sequenzialmente la lettura, l'elaborazione e la risposta.

#### Motivazioni e Trade-off:

- Semplicità del modello concorrente: garantisce un flusso di esecuzione lineare per ogni client.
- L'attributo *detached* previene i *memory leak*, in quanto lo stack del thread viene automaticamente riassorbito dal sistema operativo alla chiusura del socket.
- Sebbene i modelli *event-driven* asincroni siano più scalabili per migliaia di connessioni, il modello thread-per-client risulta adeguato e più rapido da implementare per il carico previsto in ambiente di laboratorio.

#### 3.2 Granularità dei Lock (Fine-Grained Locking)

A differenza di approcci centralizzati che utilizzano un unico lock globale per l'intera struttura dati, il nostro sistema adotta un approccio *Fine-Grained*. L'accesso in mutua esclusione è garantito da un array di `pthread_mutex_t`, uno per ogni singola partita (`lock_partita`). Questa specifica scelta implementativa massimizza il parallelismo: l'elaborazione di una mossa nella "Partita 1" blocca esclusivamente il lock di quella stanza, consentendo ai thread della "Partita 2" di operare in totale concorrenza senza subire colli di bottiglia causati da un mutex globale.

#### **4. Macchina a Stati della Partita**

Il ciclo di vita di una stanza è modellato come una Macchina a Stati Finiti (FSM) esplicita. I macro-stati adottati sono LIBERA, IN\_ATTESA, IN\_ACCETTAZIONE, IN\_CORSO e TERMINATA.

##### **Motivazioni:**

- Riduce i comportamenti impliciti: ogni stringa di rete ricevuta viene validata rispetto allo stato corrente. Ad esempio, un comando MOVE ricevuto nello stato IN\_ACCETTAZIONE viene tacitamente ignorato, schermando il server da pacchetti anomali.
- Facilita il ragionamento su casi limite e transizioni concorrenti.

## 5. Sincronizzazione Asincrona (Il Problema del Consenso)

Il progetto richiedeva l'implementazione di dinamiche in cui due client dovevano raggiungere un accordo (l'approvazione di ingresso e la rivincita) senza bloccare le risorse di rete.

- **Meccanismo di Accettazione ("Citofono"):** La transizione da IN\_ATTESA a IN\_CORSO non è diretta. Quando lo sfidante tenta l'accesso, il sistema transita in IN\_ACCETTAZIONE. Il thread dello sfidante viene sospeso in attesa, mentre il server notifica asincronamente il creatore in attesa di un SI/NO esplicito.
- **Rivincita a Doppio Consenso:** Per evitare che un singolo giocatore forzi l'azzeramento della griglia, la transizione da TERMINATA a IN\_CORSO richiede l'accumulo di due comandi REMATCH. Solo al raggiungimento dell'unanimità il server applica le modifiche all'area di memoria condivisa. Un rifiuto (NO\_REMATCH) forza invece l'abbattimento della stanza in modo sicuro.

## 6. Architettura del Broadcast Globale

Per soddisfare il requisito della "Bacheca", si è optato per il disaccoppiamento concettuale tra la *rete di gioco* e la *rete di sistema*. Il server detiene un array dinamico globale di file descriptor corrispondenti a tutti i client connessi. Quando avviene un cambio di stato rilevante (creazione o termine partita), un'apposita routine itera in mutua esclusione su tale array, escludendo i giocatori direttamente coinvolti nella partita, e invia in multicast un messaggio NOTIFICA. Questo pattern disaccoppia la reportistica di log dal flusso critico dei socket impegnati nel gameplay.

## 7. Protocollo Applicativo Testuale

Il protocollo applicativo è basato su messaggi testuali delimitati, con payload minimali. I pacchetti differenziali (es. UPDATE;RIGA;COLONNA) evitano la trasmissione dell'intera matrice di gioco.

### Motivazioni:

- La scelta del formato testuale riduce la complessità implementativa lato server in C.
- Semplifica enormemente l'operazione di *parsing* (condotta tramite i metodi nativi di scomposizione delle stringhe nei rispettivi linguaggi).

## 8. Containerizzazione e Operatività

Il backend del sistema è containerizzato tramite Docker Compose. Questo assicura l'isolamento dell'ambiente di compilazione GCC e del demone in esecuzione, astruendo le differenze architetturali del sistema host. Il server espone la propria porta di ascolto agganciandosi alla rete dell'host locale, pronto ad accettare connessioni dai client nativi.

## 9. Paradigma Event-Driven del Client (Java Swing)

Il linguaggio Java è stato scelto per il suo modello di threading ad alto livello e per la gestione esplicita delle eccezioni di rete (es. `IOException`). L'uso di un ecosistema differente dal backend rafforza la separazione architetturale, forzando un'interazione basata unicamente sugli standard TCP.

Il client non opera in modo bloccante procedurale, bensì adotta un paradigma Reattivo/Event-Driven. Un thread di background si occupa esclusivamente dell'I/O bloccante sul socket. Non appena viene intercettato un pacchetto, il payload viene decodificato e inoltrato all'Event Dispatch Thread (EDT) di Java tramite il metodo `SwingUtilities.invokeLater`. Questo garantisce che l'aggiornamento della griglia avvenga in modo *thread-safe* e senza mai causare il congelamento (freeze) dell'interfaccia utente.

## 10. Conclusione

Il sistema adotta una strategia pragmatica fortemente basata su un server autoritativo. Le proprietà di correttezza e la riduzione delle *race condition* derivano direttamente dalle scelte architetturali proattive (isolamento con *fine-grained locking* e flussi asincroni) piuttosto che da meccanismi reattivi di recovery. Tali soluzioni privilegiano coerenza, scalabilità parallela e prevedibilità operativa nel contesto del laboratorio.